

京阿尼原画管理系统功能报告

一、代码概述

- 1.本项目使用 C++作为主要的编程语言。
- 2.本项目使用的是 mysql 数据库管理系统，所有 SQL 语法遵循 mysql 的规定。
- 3.本工程的编写环境是 Visual Studio2017，因为工程太大，只拉取出必要的关键文件。解决方案配置为“Debug”，解决方案平台为“x64”。文件结构如下：

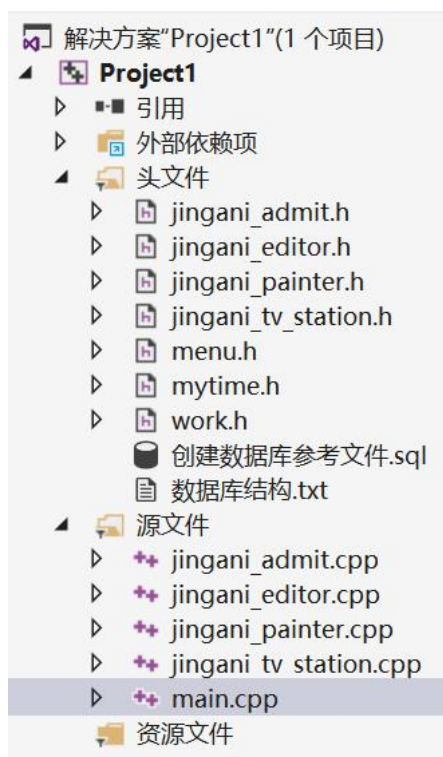


图 1 VS 工程文件结构

其中，main 函数，即工程入口在 main.cpp 里。具体文件的调用关系参见“三、代码架构”。

二、连接和使用数据库的方法

为了连接数据库，需要做以下几件工作：

- (1) 获取用户名、密码、IP、数据库等信息
- (2) 通过相关语句连接数据库

通过上网查阅相关数据，我找到了一种用 C++进行编程的方法。相较于 SQL Server，mysql 的体积更加小巧，并且我们可以从其安装位置找到相关的 include

和 lib 目录，我们将 mysql.h 作为头文件包含在我们的代码中，就可以进行接下来的操作，而且可以将一些必要的函数进行进一步的封装。主要的一些函数如下：

1. mysql_real_connect()

函数原型：

```
MYSQL *mysql_real_connect(MYSQL *mysql, const char *host, const char *user,
const char *passwd, const char *db, unsigned int port, const char *unix_socket,
unsigned int client_flag)
```

重要参数说明：

第一个参数应该是一个现存 MYSQL 结构的地址。在调用 mysql_real_connect() 之前，你必须调用 mysql_init() 初始化 MYSQL 结构。host 值可以是一个主机名或一个 IP 地址。如果 host 是 NULL 或字符串 "localhost"，假定是到本地主机的一个连接。user 参数包含用户的 MySQL 登录 ID。如果 user 是 NULL，假定是当前用户。passwd 参数为 user 包含口令。db 是数据库名。

函数返回值：

如果连接成功，则返回一个 MYSQL* 连接句柄。如果连接失败，返回 NULL。

2. mysql_select_db()

函数原型：

```
int mysql_select_db(MYSQL *mysql, const char *db)
```

重要参数说明：

使得由 db 指定的数据库成为 在由 mysql 指定的连接上的缺省(当前)数据库。在随后的查询中，这个数据库对于不包括一个显式的数据库指定符的表的引用是缺省数据库。除非连接的用户能被认证允许使用数据库，否则 mysql_select_db() 失败。

函数返回值：

成功，则返回零。如果发生一个错误，则返回非零。

3. mysql_real_query()

函数原型：

```
int mysql_real_query(MYSQL *mysql, const char *query, unsigned int length)
```

重要参数说明：

执行由 query 指向的 SQL 查询，它应该是一个 length 个字节的字符串。查询必须由一个单个的 SQL 语句组成。

函数返回值：

如果查询成功，返回零。如果发生一个错误，返回非零。

4.mysql_store_result()

函数原型：

MYSQL_RES *mysql_store_result(MYSQL *mysql)

重要参数说明：

用于将 mysql_real_query 查询结果返回

函数返回值：

返回 MYSQL_RES 结构，如果获取失败则返回空

5.mysql_fetch_row()

函数原型：

MYSQL_ROW *mysql_fetch_row(MYSQL_RES* res)

重要参数说明：

用于读取 MYSQL_RES

函数返回值：

返回表示 MYSQL_RES 的下一行的 MYSQL_ROW

三、代码封装

为了减少程序的冗余，我们可以将 mysql.h 中一些原始的函数进行封装，这样，会简化我们的操作，我们的工作重心就转移到使用生成合法的 SQL 语句和正确性检验上了。我们有如下封装：

1.连接 mysql 与数据库部分

如下图所示，如下结构即可以实现连接数据库：

```
string dbuser = "root";
string dbpasswd = "111456aa";
string dbip = "localhost";
string database = "jingani";
//上述信息需要修改

MYSQL *con = mysql_init((MYSQL*) 0);
if (con != NULL && mysql_real_connect(con, dbip.data(), dbuser.data(), dbpasswd.data(), database.data(), 3306, NULL, 0))
{
    if (!mysql_select_db(con, database.data()))
    {
        cout << "成功连接数据库,系统可以开始使用"<<endl;
        con->reconnect = 1;
    }
}
```

图 2 连接 mysql 与 jingani 数据库部分

需要注意的是，MYSQL*类型的 con 是个极其重要的参数，它将被几乎所有的函数直接或者间接的调用，整个工程都在使用同一个 con。

2.普通查询部分

如下图所示：

```
string tmp;  
//<tmp>写入sql语句  
size_t rt = mysql_real_query(con, tmp.data(), tmp.size());  
if (!rt)  
{  
    cout << "查询表达式正确" << endl;  
}  
else  
{  
    cout << "查询表达式错误" << endl;  
}
```

图3 普通查询部分

如果查询正确，则进入 if (!rt)分支，否则，进入 else 分支并结束。这是最常用的部分，其中 if (!rt)分支中还可以进行其他的操作。

3.select 查询输出结果部分

如下图所示：

```
string tmp;  
//<tmp>写入select语句  
size_t rt = mysql_real_query(con, tmp.data(), tmp.size());  
if (!rt)  
{  
    MYSQL_RES *res = mysql_store_result(con);  
    MYSQL_ROW row;  
    while (row = mysql_fetch_row(res))  
    {  
        for (int t = 0; t < mysql_num_fields(res); t++)  
        {  
            if (row[t] == NULL)  
            {  
                cout << " ";  
            }  
            else  
            {  
                cout << row[t] << " ";  
            }  
        }  
        cout << endl;  
    }  
}  
else  
{  
    cout << "查询表达式错误" << endl;  
}
```

图4 select 查询输出结果部分

在普通查询部分的 if (!rt)分支中逐一获取查询结果，结果也可以被 string 等数据类型接收，比较方便。

考虑到上述结构比较庞大，可以进一步封装：

```
void print(MYSQL_RES *res, MYSQL *con)  
{  
    MYSQL_ROW row;  
    while (row = mysql_fetch_row(res))  
    {  
        for (int t = 0; t < mysql_num_fields(res); t++)  
        {  
            if (row[t] == NULL)  
            {  
                cout << " ";  
            }  
            else  
            {  
                cout << row[t] << " ";  
            }  
        }  
        cout << endl;  
    }  
}
```

图5 print 函数

```

string tmp;
//对tmp写入select语句
size_t rt = mysql_real_query(con, tmp.data(), tmp.size());
if (!rt)
{
    MYSQL_RES *res = mysql_store_result(con);
    print(res, con);
}
else
{
    cout << "查询表达式错误" << endl;
}

```

图 6 封装后的 select 语句输出结果部分

需要注意的是，因为本工程实际使用面向对象的编程方式，所以 print 充当一个成员函数，结构也有所改变。

4.update 封装

考虑到本系统实际上使用最多的语句只有 select 和 update，而且在 update 的 if (!rt)分支中并不会进行多余的操作，所以把 update 的语句单独拿出来封装是合适的，我们有如下封装：

```

bool to_update(string tmp, MYSQL *con)
{
    size_t rt = mysql_real_query(con, tmp.data(), tmp.size());
    if (!rt)
    {
        return true;
    }
    else
    {
        cout << "表达式错误" << endl;
        return false;
    }
}

```

图 7 封装后的 update

需要注意的也是，此封装会根据具体情况做出改变。

四、代码架构

1.代码的相互包含关系

1.1main.cpp

对于 main.cpp，除了一些常用的头文件，还需要包含：

（1）mysql 相关头文件。包括 “#include"mysql.h ” 和 “ #pragma comment(lib,"libmysql.lib")”

（2）“work.h” 即工作头文件。

main.cpp 是集成度最高的层，里面只进行简单的选择，所有的实现都放在 work.h 里。

1.2 work.h

对于 work.h，头文件列表如下：

```
#include<iostream>
#include<string>
#include"mysql.h"

#include"jingani_admit.h"
#include"jinganiPainter.h"
#include"jingani_editor.h"
#include"jingani_tv_station.h"

#include"mytime.h"

#include"menu.h"

#pragma once
```

图 8 work.h 头文件列表

它包含了其他所有的文件，它是在这些头文件的基础上搭建的，是集成度次高的层，只涉及功能选择和获取数据，不涉及 SQL 语句的产生。

1.3 menu.h

menu.h 是最简单的文件，它的所有功能就是输出菜单提示信息，每个函数只有 cout 一种语句，其中的函数穿插在 main.cpp 和 work.h 中，之后的函数功能也不涉及 menu.h 的介绍

1.4 mytime.h

mytime.h 里定义了 mytime 类，用以时间的处理，所有有关时间的计算都在这个类中完成。

1.5 其他的文件——四个类的定义与实现

其他的文件都有关于本项目中四个主要的类的定义与实现，四个类分别为：

- (1) jingani_admit 类，即京阿尼管理员类，有关管理员的操作在这里实现
- (2) jinganiPainter 类，即京阿尼原画画师类，有关原画画师的操作在这里实现
- (3) jingani_editor 类，即京阿尼编辑类，有关编辑的操作在这里实现
- (4) jingani_tv_station 类，即电视台管理员类，有关电视台管理员的操作在这里实现

这四个类是本项目的最底层实现，负责根据 work.h 传入的参数生成合法的 SQL 语句并进行查询。

2.代码流程

2.1main.cpp 流程图

main.cpp 流程图如下：

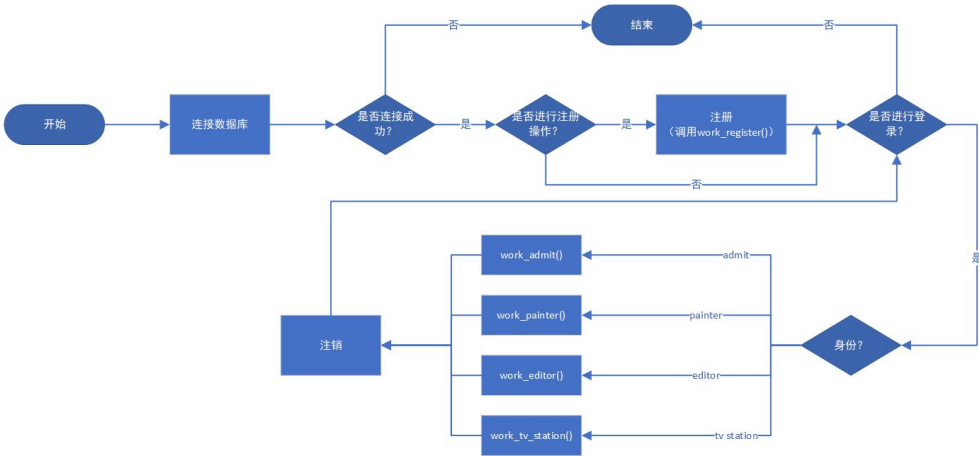


图 9 main.cpp 流程图

2.2work 函数通用流程图

work_admit()、workPainter()、work_editor()和 work_tv_station()四个函数有通用的模板，虽然功能差别很大，我们将四个函数抽象成 work_common()函数，并替换相关对象，可以得到 work_common()的流程图：

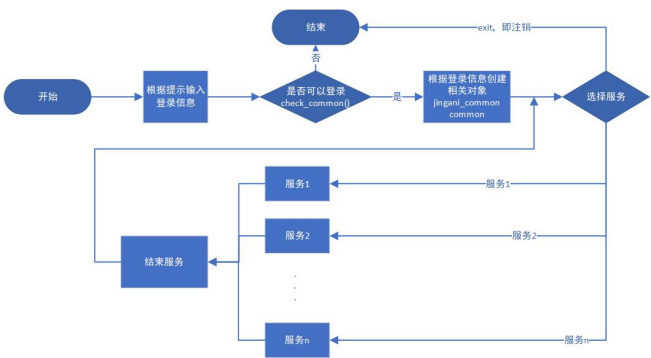


图 10 work_common() 流程图

具体每个函数的功能可以参考使用报告来具体化。不再对每个具体服务进行流程图的细化。

五、功能说明

0.说明

(1) 为了叙述的方便，本部分先简要介绍 main 的实现，之后根据 work 提

供的服务顺序来进行说明，在介绍服务的过程中，即可以将各个类的成员函数介绍清楚。

(2) `main.cpp` 的实现比较简单，简单抽象后如下，可以根据流程图简单实现，只需要增加一些判断和循环即可

(3) `work.h` 中不介绍 `work_test()`，这个是方便测试而增加的，也不介绍 `work_register()`，因为实现比较简单。

每个类定义中都存在四个私有成员变量：

```
private:
    MYSQL* con;
    MYSQL_RES *res;
    MYSQL_ROW row;
    string name;
```

图 11 私有成员变量

其中 `con` 和 `name` 在创建对象时即传入，其余两个成员是方便简化代码使用，因为是在成功登录后才创建成员，所以不需要保存密码。

(4) 在介绍每个底层函数时，不涉及到具体的 SQL 语句（即某一具体值的查询），用形如“`select * from jingani.table where x='y'`”的语句来表示，其中 `table` 代表一个表，`x` 代表一个属性，`y` 代表一个具体值。

1.mytime 类的定义实现

`mytime` 类处理所有有关时间的信息。

因为数据库中所有使用到的时间的数据类型都为“`datetime`”，数据格式为“`XXXX-XX-XX XX:XX:XX`”从前到后依次代表年、月、日、小时、分、秒。“`datetime`”在 SQL 中可以直接比较大小。

1.1mytime 类的定义

`mytime` 类的定义如下：

```
class mytime
{
private:
    time_t now;
    tm local;
public:
    //构造函数
    mytime() {};
    mytime(string datetime);
    mytime(int year, int month, int day, int hour, int minute, int second);

    //时间设置函数
    void set_time(string datetime);
    void set_time(int year, int month, int day, int hour, int minute, int second);

    //获得值函数
    string get_datetime(); //获取当前时间（以datetime格式返回）
    string get_tm(); //获取类中存储时间（以datetime格式返回）
    int get_day(); //获取本月天数
    int get_last_day(); //获取上个月天数

    //时间平移函数
    bool putoff_time(int year, int month, int day, int hour, int minute, int second); //时间推移
    bool advance_time(int year, int month, int day, int hour, int minute, int second); //时间提前

    //友元函数，时间比较
    friend bool operator>(const mytime a, const mytime b);
};
```

图 12 mytime 类定义

其中，结构体 `tm` 的结构为：

```
/*struct tm {
    int tm_sec;    // 秒，正常范围从 0 到 59，但允许至 61
    int tm_min;    // 分，范围从 0 到 59
    int tm_hour;   // 小时，范围从 0 到 23
    int tm_mday;   // 一月中的第几天，范围从 1 到 31
    int tm_mon;    // 月，范围从 0 到 11
    int tm_year;   // 自 1900 年起的年数
    int tm_wday;   // 一周中的第几天，范围从 0 到 6，从星期日算起
    int tm_yday;   // 一年中的第几天，范围从 0 到 365，从 1 月 1 日算起
    int tm_isdst;  // 夏令时
}*/
```

图 13 结构体 `tm` 结构

这是主要用来操作的结构体。

1.2 mytime 主要函数的实现

(1) 构造函数 `mytime(string datetime)`

本系统中所有的有关时间的字符串都和“`datetime`”数据类型一致，使用 `sscanf_s` 函数就可以完成数据的输入。如下图所示，其他的值不需要考虑：

```
mytime::mytime(string datetime)
{
    //wait
    sscanf_s(datetime.data(), "%d-%d-%d %d:%d:%d",
        &local.tm_year, &local.tm_mon, &local.tm_mday, &local.tm_hour, &local.tm_min, &local.tm_sec);
    local.tm_year -= 1900;
    local.tm_mon -= 1;
    //我们不需要其他属性
}
```

图 14 `mytime(string datetime)`

(2) 获取当前时间并输出函数

获取当前日期需要涉及到一些系统调用了，经过对比，这里找到一种较为准确的获得当前时间的方法，它最终返回一个 `tm` 类型的变量。再通过 `spaintf_s` 就可以将 `tm` 类型的变量转为一个“`datetime`”类型的字符串。返回现在存储的时间的函数也是一样的操作，只不过是少了获取当前时间的步骤。

```
string mytime::get_datetime()
{
    //cout << "now=time(NULL)";
    //now = time(NULL);
    time(&now);
    //cout << "localtime" << endl;
    localtime_s(&local, &now);
    char tmp[50];
    sprintf_s(tmp, "%4d-%02d-%02d %02d:%02d:%02d",
        1900+local.tm_year, local.tm_mon+1, local.tm_mday, local.tm_hour, local.tm_min, local.tm_sec);
    string res(tmp);
    //cout << res;
    return res;
}
```

图 15 `get_datetime()`

(3) 时间延期与提前函数

这两个函数复杂的地方在于，时间的进位是不统一的，每个月都有不同的天数。而两个函数的输入都是从年到秒，如何处理进位问题是一大难题。

于是，经过设计，有如下的流程：

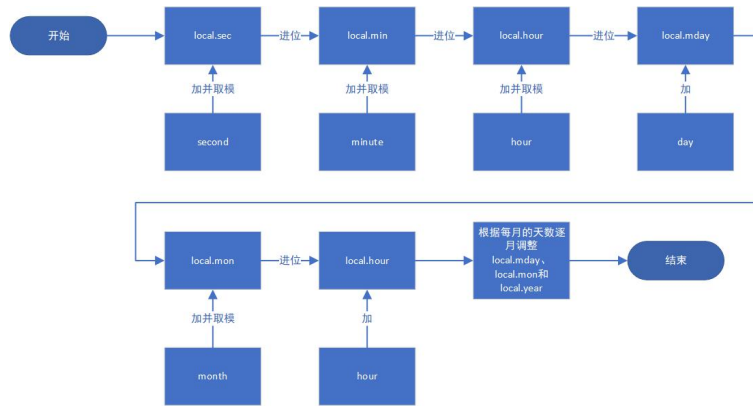


图 16 时间推迟流程

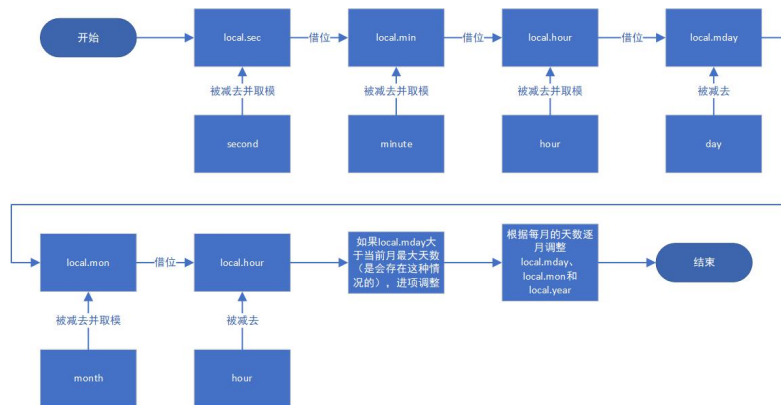


图 17 时间提前流程

2.登录函数的实现

登录函数是独立于四个主要类之外的，他在创建对象之前使用。

根据用户输入的信息，SQL 语句如下：

```
select * from jingani.common where name='name' and password='password'
```

其中，common 是 painter、editor、admit、tv_station 中的一个。如果有返回，则认为登陆成功。

3.管理员服务函数

管理员目前只有一个上帝视角服务。在这里，管理员可以输入任何合法的 SQL 语句，在将 SQL 语句送到相关函数之前，会对 SQL 语句做一定的判断，如果 SQL 语句以 select 开头，则需要打印操作。

4.画师服务

根据提供给画师的服务逐一介绍：

4.1mode check（根据模式查询审查表信息）

只需要调用一个函数即可完成。函数作用是在 `check` 表中寻找符合条件的审查信息，其中，必有的条件是画师姓名和当前登录的画师姓名相同，根据查询的模式的不同，还可以根据原画的审核状态进行细化查询。

函数原型：

```
void jinganiPainter::mode_check(string mode)
```

对应的 SQL 语句：

all 模式：

```
select * from jingani.check where Painter_name='name'
```

其他模式：

```
select * from jingani.check where Painter_name='name' and Status='mode'
```

4.2find editors（查找不同状态的编辑）

只需要调用一个函数即可完成。函数作用是在 `editor` 表中寻找符合条件的审查信息，根据查询的模式的不同，还可以根据编辑的状态进行细化查询。

其中，`free` 模式需要考虑到编辑的当前审查原画数目要小于最大可审核原画数目；`reserve` 模式需要考虑到编辑的可预约原画数目应该大于 0。

函数原型：

```
void jinganiPainter::select_edit(string mode)
```

对应的 SQL 语句：

all 模式：

```
select Name,Sex,max_edit,now_edit,queue_edit,Level from jingani.editor
```

free 模式：

```
select Name, Sex, max_edit, now_edit, queue_edit, Level from jingani.editor  
where now_edit<max_edit
```

reserve 模式：

```
select Name, Sex, max_edit, now_edit, queue_edit, Level from jingani.editor  
where queue_edit>0
```

4.3delievry（交付原画）

共需要四个函数来完成此服务，其中，两个是合法性判断函数，两个是状态修改函数。

首先，系统需要获取提交的编辑姓名、原画编号和原画来源，然后进行判断。

第一个函数是检查编辑目前是否是空闲状态，即编辑的当前审查原画数目要小于最大可审核原画数目。

第二个函数是检查原画编号是否合法，即不允许在审查表中出现重复的原画

编号。

仅当前两个函数都通过（都返回 true）时才进行真正的状态修改。

第一个状态修改函数的作用是在 check 表中添加一条审查记录。

第二个状态修改函数的作用是修改对应的编辑属性，即当前审核原画数目加一。

函数原型：

- (1) bool jinganiPainter::check_editor(string editor, string mode)
- (2) bool jinganiPainter::check_num(string num)
- (3) void jinganiPainter::add_check(string editor, string num, string source, string status)
- (4) void jinganiPainter::change_editor(string editor, string status)

对应的 SQL 语句：

- (1) select * from editor where Name='name' and now_edit<max_edit（这里 mode 选择 free）（当查询有结果返回时函数返回真）
- (2) select * from jingani.check where ori_p_num='name'（当查询无结果返回时函数返回真）
- (3) insert into jingani.check (ori_p_num, Edit_name, Source, Painter_name, Status, instorage) values('num', 'editor', 'source', 'name', 'ing', 0)（这里插入即为正在审核状态）
- (4) update editor set now_edit=now_edit+1 where Name='editor'（status 选择 delivery）

4.4reserve（预约原画）

和交付原画流程很像。只需要修改相关的状态就可以，最后的状态修改函数是使编辑可预约原画减一。

函数原型：

- (1) bool jinganiPainter::check_editor(string editor, string mode)
- (2) bool jinganiPainter::check_num(string num)
- (3) void jinganiPainter::add_check(string editor, string num, string source, string status)
- (4) void jinganiPainter::change_editor(string editor, string status)

对应的 SQL 语句：

- (1) select * from editor where Name='name' and queue_edit>0（这里 mode 选择 reserve）（当查询有结果返回时函数返回真）
- (2) select * from jingani.check where ori_p_num='name'（当查询无结果返回时函数返回真）
- (3) insert into jingani.check (ori_p_num, Edit_name, Source, Painter_name, Status, instorage) values('num', 'editor', 'source', 'name', 'wait', 0)（这里插入即为待审核状态）

(4) update editor set queue_edit=queue_edit-1 where Name='editor' (status 选择 delivery)

4.5re_delievry (重新交付原画)

只需要调用一个函数即可完成。此函数会进行进一步的调用。函数的作用是 根据输入的原画编号，先进行检验此原画的审核状态是否是“false”，如果是，则继续判断当前的编辑是否是 free 状态，如果是，则将原画的审核状态改为“ing”（实际上，这两个步骤使用嵌套查询一次完成），并且修改对应的编辑属性，即当前审核原画数目加一。

不允许重新交付时为 wait 状态。

函数原型：

```
void jinganiPainter::re_check(string num)
```

对应的 SQL 语句：

```
select * from jingani.editor where now_edit<max_edit and Name in (select Edit_name from jingani.check where ori_p_num='num' and status='false');
```

```
select Edit_name from jingani.check where ori_p_num='num' and status='false' (在此步骤获取编辑姓名，方便后面对编辑状态进行修改)；
```

```
update editor set now_edit=now_edit+1 where Name='editor' (status 选择 delivery)；
```

```
update jingani.check set Status='ing' where ori_p_num='name'
```

4.6my paint instroage (查询自己已经入库的原画的信息)

只需要调用一个函数即可完成。函数作用是在 ori_paint 表中查找所有当前登录的画师的入库的原画。

函数原型：

```
void jinganiPainter::paint_in_store()
```

对应的 SQL 语句：

```
select * from jingani.ori_paint where Painter_name='name'
```

5.编辑服务

根据提供给编辑的服务逐一介绍：

5.1check task (根据模式查询自己的审查任务)

只需要调用一个函数即可完成。函数作用是在 check 表中寻找符合条件的审查信息，其中，必有的条件是编辑姓名和当前登录的编辑姓名相同，根据查询的

模式的不同，还可以根据原画的审核状态进行细化查询。

函数原型：

```
void jingani_editor::check_task(string status)
```

对应的 SQL 语句：

all 模式：

```
select ori_p_num, Source, Painter_name, Status from jingani.check where Edit_name='name'
```

ing 和 wait 模式：

```
select ori_p_num, Source, Painter_name, Status from jingani.check where Edit_name='name' and Status='status'（status 可选择 “ing” 或者 “wait”）
```

storage 模式：

```
select ori_p_num, Source, Painter_name, Status from jingani.check where Edit_name='name' and Status='pass'and instorage=0
```

5.2take（接收预约的原画）

需要四个函数完成。

第一个函数是编辑状态检查函数，只有当编辑当前审核原画数目小于最大可审核原画数目，编辑才可以进行接收预约的原画操作。

第二个函数自动执行，对预约的原画信息进行查询并打印出来。

第三个函数是判断函数，判断输入的原画信息是否在可预约的原画里。

第四个函数是状态修改函数，将修改审核信息（将目标原画的记录的状态由“wait”改为“ing”）和编辑信息（编辑审核原画数目加一、可预约原画减一）。

函数原型：

(1) bool jingani_editor::can_take()

(2) void jingani_editor::check_task(string status)

(3) bool jingani_editor::check_num(string num, string status)

(4) void jingani_editor::take_task(string num)

对应的 SQL 语句：

(1) select * from editor where Name='name' and now_edit<max_edit

(2) select ori_p_num, Source, Painter_name, Status from jingani.check where Edit_name='name' and Status='wait'（选择 wait 模式）

(3) select * from jingani.check where ori_p_num='num' and Status='wait'（选择 wait 模式）

(4) update jingani.check set Status='ing' where ori_p_num='num';

update editor set now_edit=now_edit+1 where Name='name';

update editor set queue_edit=queue_edit+1 where Name='name'

5.3check（审核原画）

需要四个函数完成，但根据编辑的不同的选择，实际运行的函数可能是三个或四个。

第一个函数自动执行，对正在审核的原画信息进行查询并打印出来。

第二个函数是对输入的原画编号进行检查，只有输入的原画编号是待审核的原画才会进行下一步操作。

两个函数完成后，需要编辑进行选择，即原画是否合格通过，仅当通过时，第三个函数运行（第四个函数都会运行）。

第三个函数的作用是对审核的原画进行打分，并且修改原画审核的分数和审核通过时间（通过 `mytime` 类自动获取操作时的时间）。

第四个函数是根据编辑的选择对原画审核状态进行修改，同时对编辑状态进行修改（编辑审核原画数目减一）。

函数原型：

- (1) `void jingani_editor::check_task(string status)`
- (2) `bool jingani_editor::check_num(string num, string status)`
- (3) `void jingani_editor::give_score(string num, string score, string datetime)`
- (4) `void jingani_editor::check(string num, string status)`

对应的 SQL 语句：

(1) `select ori_p_num, Source, Painter_name, Status from jingani.check where Edit_name='name' and Status='ing'`（选择 ing 模式）

(2) `select * from jingani.check where ori_p_num='num' and Status='ing'`（选择 ing 模式）

(3) `update jingani.check set Quality='score' where ori_p_num='num';`

`update jingani.check set pass_time='datetime' where ori_p_num='num'`

(4) `update editor set now_edit=now_edit-1 where Name='name';`

审核成功：

`update jingani.check set Status='pass' where ori_p_num='num'`

审核失败：

`update jingani.check set Status='false' where ori_p_num='num'`

5.4storage（入库原画）

第一个函数自动执行，对已经审核通过且还没有入库的原画信息进行查询并打印出来。

之后分为两种模式，`one` 模式和 `all` 模式。`all` 模式将自动将所有已经审核通过且还没有入库的原画进行入库操作，其只需要一个函数，将在 `one` 模式后介绍。

如果选择 `one` 模式，则接下来还需要四个函数。其中第二个函数（包括上面提到的函数）是判断函数，其余三个都是状态修改函数，分别对 `check`

第二个函数是对输入的原画编号进行检查，只有输入的原画编号是已经审核通过且还没有入库的原画才会进行下一步操作。

第三个函数是增加入库信息函数。即在 `storage` 中增加一条记录。这里需要注意的是，因为外键选择的原因，必须先进行此步操作才能增加入库原画信息。

第四个函数是增加入库原画信息函数。为了完成这一步操作，需要先在 `check` 表中获取相关的原画信息，并通过其中的画师信息去 `painter` 表中获取画师的作画风格信息，然后才能执行插入操作。

第五个函数是其他状态修改函数。作用是将原画审核状态中的 `instorage` 属性值更改为 1（表示已经入库）。

one 模式函数原型：

- (1) `void jingani_editor::check_task(string status)`
- (2) `bool jingani_editor::check_num(string num, string status)`
- (3) `void jingani_editor::insert_storage(string num, string datetime)`
- (4) `void jingani_editor::to_instore(string num, string datetime)`
- (5) `void jingani_editor::ready_instore(string num)`

one 模式对应的 SQL 语句：

- (1) `select ori_p_num, Source, Painter_name, Status from jingani.check where Edit_name='name' and Status='pass' and instorage=0`（选择 `storage` 模式）
- (2) `select * from jingani.check where ori_p_num='num' and Status='pass' and instorage=0`（选择 `storage` 模式）
- (3) `insert into jingani.storage values('num', 'name', 'datetime')`
- (4) `select Painter_name, Source, Quality from jingani.check where ori_p_num='num';`（这一步的目的从 `check` 中获取需要插入的信息）
`select Style from painter where Name='painter_name';`（这一步的目的是从 `painter` 中获取画师的作画风格，认为画师的作画风格即为原画的风格）
`insert into jingani.ori_paint values('num', 'source', 'painter_name', 'style', 'quality')`
- (5) `update jingani.check set instorage=1 where ori_p_num='num'`

对于 `all` 模式，即先通过查询获取所有已经审核通过且还没有入库的原画编号并逐个进行 `one` 模式操作，需要注意的是，这时候入库的所有原画的入库时间都是一样的（因为只从 `mytime` 类中获取时间一次）。

all 模式函数原型：

- (1) `void jingani_editor::check_task(string status)`
- (2) `void jingani_editor::to_instore_all(string datetime)`

all 模式对应的 SQL 语句：

- (1) `select ori_p_num, Source, Painter_name, Status from jingani.check where Edit_name='name' and Status='pass' and instorage=0`（选择 `storage` 模式）
- (2) `select ori_p_num from jingani.check where Edit_name='name';`（这一步的目的是得到所有的待入库原画编号，之后对每一个编号都执行 `one` 模式对应的 SQL 语句的 (3) (4) (5)，在这里不需要再对原画编号进行判断）

6.电视台管理员服务

根据提供给电视台管理员的服务逐一介绍：

6.1check list（查询所有的播放计划）

只需要一个函数即可实现。对 tv_list 的本电视台的所有内容进行查询即可。

函数原型：

```
void jingani_tv_station::check_list_all()
```

对应的 SQL 语句：

```
select * from tv_list where TV_name='name'
```

6.2check anime（查找某一番剧的播放计划）

也是只需要一个函数即可实现。对 tv_list 的本电视台播放的某一番剧所有内容进行查询即可，为了查询结果符合人的认知，将会对查询结果按照集数升序排列。

函数原型：

```
void jingani_tv_station::check_anime(string anime)
```

对应的 SQL 语句：

```
select * from tv_list where TV_name='name' and Anime='anime' order by Episode
```

6.3add list（增加一条本电视台的播放计划）

只需要一个函数即可实现。根据输入的信息在 tv_list 表中插入一条记录即可。在插入的时候会对时间的合法性进行检验，即不允许同一时间有多于一个播放计划。

函数原型：

```
void jingani_tv_station::add_a_list(string anime, string source, int episode, string start_time, string end_time)
```

对应的 SQL 语句：

（1）select * from jingani.tv_list where TV_name='name' and start_time <'start_time' and end_time>'start_time';（这一步的目的是对输入的播放开始时间的合法性进行初步检验，即开始时间不能落在另一播放计划的区间中）

（2）select * from jingani.tv_list where TV_name='name' and start_time <'end_time' and end_time>'end_time';（这一步的目的是对输入的播放结束时间的

合法性进行初步检验，即开始时间不能落在另一播放计划的区间中)

(3) select * from jingani.tv_list where TV_name='name' and start_time>'start_time' and end_time<'end_time'; (这一步的目的是对输入的播放时间的合法性进行再次检验，此播放计划时间段不能在全部某一播放计划的时间段内)

(4) select * from jingani.tv_list where TV_name='name' and start_time<'start_time' and end_time>'end_time'; (这一步的目的是对输入的播放时间的合法性进行再次检验，某一播放计划的时间段不能全部在此播放计划时间段内)

(5) insert into jingani.tv_list values('name', 'anime', 'source', epi, 'start_time', 'end_time')

为了方便起见，将 (1) - (4) 的 SQL 统称为**时间合法性检验**。

6.4change list (修改一条本电视台的播放计划)

只需要一个函数即可实现。根据输入的信息在 tv_list 表中直接修改播放时间。

函数原型：

```
void jingani_tv_station::change_time(string anime, string source, int episode, string start_time, string end_time)
```

对应的 SQL 语句：

(1) 时间合法性检验

(2) update jingani.tv_list set start_time='start_time' where TV_name='name' and Anime='anime' and Source='source' and Episode='epi'

(3) update jingani.tv_list set end_time='end_time' where TV_name='name' and Anime='anime' and Source='source' and Episode='epi'

6.5put off list (推迟播放某一播放计划) 与 advance list (提前播放某一播放计划)

只需要一个函数即可实现，在实现之前，需要通过 mytime 对时间进行操作。

函数原型和对应的 SQL 语句和 6.4 相同 (实际上，只是在调用函数之前进行了对时间的操作)。

6.6check with time (查询某一时间段内自己电视台的播放计划)

只需要一个函数即可实现，对输入时间段内的所有播放计划进行查询 (查询的时间段是播放计划的开始的时间段)。在查询前，会对输入时间段的合法性进行检验。查询结果会根据播放开始时间升序排列。

函数原型：

```
void jingani_tv_station::check_time(string start_time, string end_time)
```

对应的 SQL 语句:

```
select * from jingani.tv_list where TV_name='name' and start_time>'start_time'  
and start_time<'end_time' order by start_time
```